Guide d'installation VPS — La Termitière (v3.2 ULTIME CLEAN / PRODUCTION REAL)

Version : 3.2
Statut : Production (réel terrain)
Type : Installation + Maintenance + Recovery
Stack : Docker + Supabase self-hosted + React + Caddy + GHCR
Dernière mise à jour : Juillet 2026

⚠️ CORRECTIONS DU GUIDE ORIGINAL — À LIRE EN PREMIER

> Un guide de déploiement initial a été fourni. **Il contenait des omissions et des inexactitudes** qui ont nécessité des corrections en cours de route. Cette section documente **ce qui était faux ou manquant** dans le guide original pour éviter de revivre ces problèmes.

#	Sujet	Ce que disait le guide original	Ce qui fonctionne réellement
1	Génération des clés Supabase	Utiliser un outil web externe ou le CLI Supabase	Utiliser les scripts **déjà inclus** dans le dépôt Supabase : `sh utils/generate-keys.sh` puis `sh utils/add-new-auth-keys.sh`
2	Clés JWT	Générer des clés symétriques (HS256) basées sur `JWT_SECRET`	Générer des clés **asymétriques (ES256)** via `add-new-auth-keys.sh`. Obligatoire pour Auth, PostgREST et Realtime
3	Déploiement Frontend	`docker compose up -d --build` (déploiement unique)	Le build local a été utilisé pour le bootstrap (Phase A). Ensuite, une architecture **CI/CD via GHCR** a été ajoutée (Phase B) nécessitant `docker-compose.prod.yml` et un fichier `.env.deploy`.
4	Node.js pour les scripts de migration	Non mentionné	**Node.js 20+** est obligatoire. Node 18 et inférieures causent des erreurs silencieuses avec `@supabase/supabase-js`

---

0. IMPORTANT (À LIRE AVANT TOUT)

Ce guide couvre 3 scénarios :

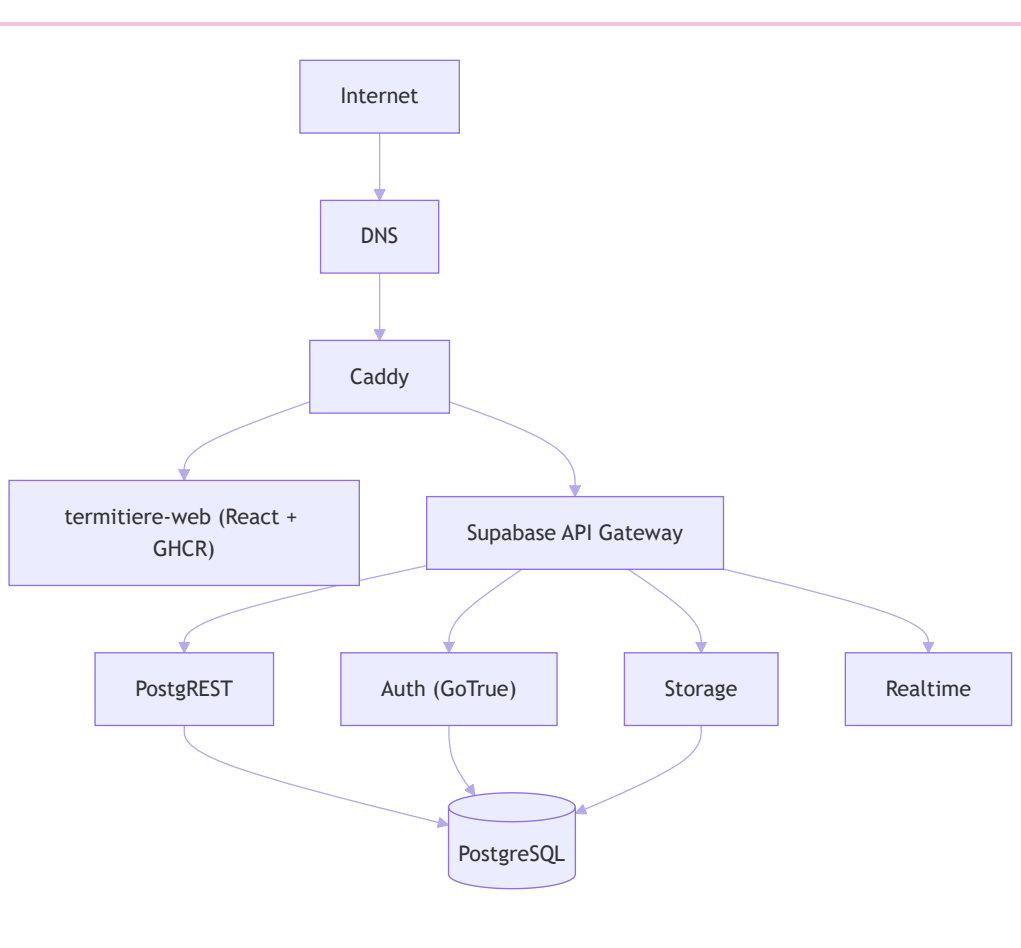
🟢 A. Nouvelle installation VPS (from scratch)

🟡 B. Installation sur VPS déjà utilisé (ton cas actuel)

🔴 C. Recovery / réparation backend Supabase

---

1. ARCHITECTURE RÉELLE



---

2. STRUCTURE VPS

```
/home/bawa/
├── termitiere-platform/     # Frontend + CI/CD
└── supabase/docker/         # Backend Supabase self-hosted
```

---

3. PRÉREQUIS VPS

Configuration réelle en production

Ressource	Valeur (VPS actuel)
CPU	4 vCPU
RAM	7,8 Go (confortéable)
Disk	148 Go
OS	Debian 13 (Trixie)

> 💡 Avec 7,8 Go de RAM, le Swap n'est **pas nécessaire**. Supabase self-hosted tourne confortablement sans. Si ton VPS n'a que 4 Go de RAM, envisager un Swap de 4 Go avant de lancer Docker.

---

4. INSTALLATION SYSTEME (BASE) & SECURITÉ

```
apt update && apt upgrade -y

apt install -y \
  curl \
  git \
  nano \
  ca-certificates \
  gnupg \
  lsb-release \
  ufw
```

4.1 Configuration Firewall (UFW)

Il est vital de fermer les ports par défaut.

```
ufw allow ssh
ufw allow http
ufw allow https
ufw enable
```

(Attention : Docker modifie iptables et peut bypasser UFW. Assure-toi que dans tes `docker-compose.yml`, les ports internes comme 5432 sont mappés sur `127.0.0.1:5432` et non `0.0.0.0:5432`).

4.2 Configuration SSH (anti-coupure) — sur ta MACHINE LOCALE

Symptôme : Après quelques minutes sans activité, ta connexion SSH au VPS se coupe avec ce message :

```
client_loop: send disconnect: Connection reset
```

Tu perds tout ce que tu étais en train de faire. La solution est d'envoyer un signal de maintien de connexion ("keep-alive") toutes les 60 secondes depuis ta machine locale.

> ⚠️ Cette configuration se fait sur TA machine (Windows/Linux/Mac), pas sur le VPS.

Sur Windows (PowerShell)

Étape 1 — Ouvrir PowerShell (touche Windows → taper `PowerShell` → Entrée)

Étape 2 — Vérifier si le dossier `.ssh` existe

```
ls $HOME\.ssh
```

- Si tu vois des fichiers → le dossier existe, passe à l'étape 3.
- Si tu vois une erreur → créer le dossier :

```
New-Item -ItemType Directory -Path "$HOME\.ssh"
```

Étape 3 — Ouvrir ou créer le fichier `config`

```
notepad $HOME\.ssh\config
```

Si Notepad te demande "Voulez-vous créer un nouveau fichier ?" → cliquer **Oui**.

Étape 4 — Coller ce contenu dans le fichier

```
Host 31.207.37.96
    ServerAliveInterval 60
    ServerAliveCountMax 10
```

Étape 5 — Sauvegarder : `Ctrl+S` → fermer Notepad.

Sur Linux / Mac (Terminal)

Étape 1 — Ouvrir le terminal

Étape 2 — Modifier ou créer le fichier de config SSH

```
nano ~/.ssh/config
```

Étape 3 — Ajouter ce bloc à la fin du fichier

```
Host 31.207.37.96
    ServerAliveInterval 60
    ServerAliveCountMax 10
```

Étape 4 — Sauvegarder : `Ctrl+X` → `Y` → `Entrée`

Vérification

Reconnecte-toi au VPS normalement :

```
ssh bawa@31.207.37.96
```

Laisse la session ouverte sans rien taper pendant 5 minutes. Si elle reste active → la configuration fonctionne.

> 💡 Que font ces paramètres ?
>
> - `ServerAliveInterval 60` : envoie un paquet "je suis toujours là" toutes les 60 secondes.
> - `ServerAliveCountMax 10` : après 10 tentatives sans réponse (soit 10 minutes), seulement alors la connexion est fermée proprement.

4.3 Authentification SSH par Clé (Sans mot de passe)

Pour une meilleure sécurité et éviter de taper ton mot de passe à chaque connexion, il est fortement recommandé d'utiliser une **paire de clés SSH** (une clé privée qui reste sur ta machine, et une clé publique que tu déposes sur le VPS).

> ⚠️ Cette opération se fait aussi depuis TA machine locale.

Étape 1 — Générer la paire de clés (sur ta machine)
Ouvre ton terminal (ou PowerShell) et tape :

```
ssh-keygen -t ed25519 -C "ton-email@exemple.com"
```

Appuie sur Entrée à toutes les questions pour accepter les choix par défaut (pas de passphrase pour un accès direct, ou ajoutes-en une pour plus de sécurité).

Cela va créer deux fichiers dans ton dossier `~/.ssh/` (ou `C:\Users\TonUser\.ssh\`) :

- `id_ed25519` : Ta clé **privée** (NE LA DONNE À PERSONNE).
- `id_ed25519.pub` : Ta clé **publique** (celle qu'on va envoyer au serveur).

Étape 2 — Copier la clé publique sur le VPS

*Si tu es sur **Linux / Mac**, utilise cette commande magique :*

```
ssh-copy-id bawa@31.207.37.96
# Il te demandera le mot de passe de "bawa" une dernière fois.
```

*Si tu es sur **Windows** (PowerShell), utilise celle-ci :*

```
$cmd = "mkdir -p ~/.ssh && chmod 700 ~/.ssh && " +
       "cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"

Get-Content $HOME\.ssh\id_ed25519.pub | ssh bawa@31.207.37.96 $cmd
```

Étape 3 — Tester la connexion sans mot de passe

```
ssh bawa@31.207.37.96
```

Si tu es connecté instantanément sans qu'il te demande de mot de passe, l'opération est un succès ! 🎉

4.4 Installation Node.js 20 (pour les scripts de migration)

⚠️ **Node.js 20 minimum est obligatoire.** La version fournie par défaut par Debian est souvent trop ancienne. Utiliser le dépôt officiel NodeSource :

```
# Ajouter le dépôt NodeSource (Node.js 20)
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs

# Vérifier la version
node --version
# → doit afficher v20.x.x ou supérieur
```

---

5. INSTALLATION DOCKER

```
curl -fsSL https://get.docker.com | sh

systemctl enable docker
systemctl start docker
```

```
apt install -y docker-compose-plugin
```

---

6. BACKEND SUPABASE (RÉEL — EXISTANT OU INSTALL)

6.1 CAS IMPORTANT : BACKEND EXISTE DÉJÀ

👉 Vérifie :

```
cd /home/bawa/supabase/docker
docker compose ps
```

Si services existants → NE PAS réinstaller sans backup !

6.2 CAS INSTALLATION PROPRE (NOUVELLE)

⚠️ **Règle d'or en prod :** Ne jamais cloner `main` à l'aveugle. Toujours fixer un tag de version (ex: v0.24.0) pour garantir la stabilité et permettre les rollbacks.

```
cd /home/bawa
git clone https://github.com/supabase/supabase.git
cd supabase
# Choisis le tag stable actuel
git checkout v0.24.0
cd docker
```

6.3 CONFIGURATION ENV BACKEND

Supabase fournit des scripts utilitaires pour générer aléatoirement et proprement **toutes** les clés de sécurité. **C'est la seule méthode valide en prod.**

```
cp .env.example .env

# 1. Génère les clés principales et injecte les valeurs dans le .env
sh utils/generate-keys.sh

# 2. Génère les clés asymétriques (ES256) et configure le JWKS
sh utils/add-new-auth-keys.sh
```

(Réponds "y" quand le script te demande de mettre à jour `.env` et `docker-compose.yml`).

💡 Que font ces scripts sous le capot ?

1. `generate-keys.sh` : Il va créer des chaînes cryptographiques fortes pour `POSTGRES_PASSWORD`, `JWT_SECRET`, `ANON_KEY`, `SERVICE_ROLE_KEY` et tous les tokens d'accès (Logflare, S3, etc.).
2. `add-new-auth-keys.sh` : Il permet le passage au **JWT asymétrique**. Il lit ton `JWT_SECRET` fraîchement créé pour générer les variables `JWT_KEYS` et `JWT_JWKS`. Cela permet aux services Auth, Realtime et PostgREST de vérifier les tokens localement sans faire de requête réseau, ce qui est critique pour les performances.

✏️ Variables à modifier MANUELLEMENT dans le .env après les scripts

⚠️ Les scripts génèrent les clés cryptographiques, mais **ces variables-là doivent être renseignées à la main**. Sans elles, le Studio sera inaccessible et l'Auth refusera les connexions.

```
nano .env
```

Variable	Valeur à mettre	Pourquoi
`SITE_URL`	`https://app.latermitiere.com`	URL de redirect après authentification
`API_EXTERNAL_URL`	`https://api.latermitiere.com`	URL publique de l'API (Kong)
`DASHBOARD_USERNAME`	Ton choix (ex: `admin`)	Accès au Studio Supabase
`DASHBOARD_PASSWORD`	Mot de passe fort	Accès au Studio Supabase
`SMTP_HOST`	(Optionnel) serveur mail	Pour les e-mails de confirmation d'Auth

Vérifier que les variables critiques sont bien renseignées :

```
grep -E 'SITE_URL|API_EXTERNAL_URL|DASHBOARD' .env
```

6.4 DÉMARRAGE BACKEND

```
docker compose up -d
```

6.5 CHECK HEALTH BACKEND

```
docker compose ps
```

Tous doivent être `healthy`. Un service "unhealthy" en production n'est **jamais normal**.

6.6 DEBUG SI PROBLÈME

```
docker logs supabase-storage
docker logs supabase-kong
docker logs supabase-auth
```

6.7 MIGRATION DES DONNÉES (ORDRE OBLIGATOIRE)

> ⚠️ **Ordre critique.** Ces étapes doivent être exécutées dans cet ordre exact. Si tu actives le RLS (`secure-auth.sql`) avant de créer les comptes, plus personne ne pourra se connecter.

1. schema.sql → 2. import.mjs → 3. auth-migrate.mjs → 4. secure-auth.sql

Étape 1 — Créer les tables (schema.sql)

```
# Depuis le dossier termitiere-platform sur le VPS
cat migration/supabase/schema.sql | \
  docker exec -i supabase-db psql -U postgres -d postgres

# Vérifier les tables créées
docker exec -i supabase-db psql -U postgres -d postgres -c '\dt public.*'
# → doit lister les tables tp_agro_*, tp_logistique_*, tp_foncier_*, etc.
```

Étape 2 — Importer les données (import.mjs)

```
cd migration/supabase
npm install

export DATABASE_URL=\
'postgresql://postgres:TON_POSTGRES_PASSWORD@localhost:5432/postgres'
node import.mjs

# → Le script affiche le nombre de lignes importées par table.
# → Vérifier dans le Studio → Table Editor que les données sont présentes.
```

Étape 3 — Créer les comptes utilisateurs (auth-migrate.mjs)

Ce script crée les comptes Supabase Auth avec des e-mails synthétiques (`@latermitiere.local`) et un mot de passe temporaire.

```
cd migration/supabase

export SUPABASE_URL='https://api.latermitiere.com'
export SUPABASE_SERVICE_KEY='TA_SERVICE_ROLE_KEY'

# Le workaround SSL est nécessaire si le certificat Caddy n'est pas encore valide
NODE_TLS_REJECT_UNAUTHORIZED=0 node auth-migrate.mjs

# → Le script affiche les comptes créés.
# → Communiquer les mots de passe temporaires aux utilisateurs.
```

⚠️ `NODE_TLS_REJECT_UNAUTHORIZED=0` est uniquement pour les outils d'admin en local. **Ne jamais l'utiliser dans le code de l'application.**

Étape 4 — Activer le RLS / Sécurité (secure-auth.sql)

⚠️ **À faire en DERNIER.** Une fois activé, seuls les utilisateurs authentifiés peuvent lire les données.

```
cat migration/supabase/secure-auth.sql | \
  docker exec -i supabase-db psql -U postgres -d postgres

# Vérifier que le RLS est actif sur les tables tp_*
docker exec -i supabase-db psql -U postgres -d postgres -c \
  "SELECT tablename, rowsecurity FROM pg_tables WHERE schemaname='public';"
# → rowsecurity doit être 't' (true) pour toutes les tables tp_*
```

✅ **Résultat final attendu :** Un utilisateur non connecté ne peut plus lire les données. Un utilisateur connecté (token JWT valide) voit uniquement ses données selon les règles RLS.

---

7. FRONTEND — DÉPLOIEMENT

> ℹ️ **Chronologie réelle :** Le premier déploiement du Frontend a été fait **manuellement** (Phase A). Une fois le système validé et stable en production, le pipeline GitHub Actions a été mis en place pour **automatiser** les déploiements suivants (Phase B). Ne pas brûler les étapes.

7.1 STRUCTURE VPS (RÉELLE)

```
/home/bawa/termitiere-platform/
├── deploy/
│   ├── Dockerfile              # Multi-Stage Build (Node 20 → Caddy)
│   ├── docker-compose.yml      # Stack Frontend (local)
│   ├── docker-compose.prod.yml # Stack Frontend (CI/CD)
│   ├── Caddyfile               # Routing SPA + Reverse Proxy API
│   └── .env                    # DOMAIN_APP, DOMAIN_API, IMAGE_TAG
├── .github/
│   └── workflows/
│       └── ghcr-test.yml       # Pipeline CI/CD automatisé
└── src/                        # Code source React
```

7.2 PHASE A — PREMIER DÉPLOIEMENT MANUEL (Bootstrap)

Le premier déploiement se fait **depuis ta machine locale** avant que le CI/CD n'existe.

Étape 1 : Cloner le dépôt sur le VPS

```
# Sur le VPS, en tant que bawa
cd /home/bawa
git clone -b vps-deploy https://github.com/la-TERMITIERE/termitiere-platform.git
cd termitiere-platform
```

Étape 2 : Créer le fichier d'environnement

```
cd deploy/
cp .env.example .env
nano .env
```

Contenu minimal du `.env` à renseigner :

```
DOMAIN_APP=app.latermitiere.com
DOMAIN_API=api.latermitiere.com
IMAGE_TAG=latest
```

Étape 3 : Construire et lancer l'image manuellement (build local)

Dans cette phase d'amorçage, on utilise le fichier `docker-compose.yml` standard, qui demande à Docker de compiler le code React directement sur le VPS.

```
# Lancement de la compilation et démarrage du conteneur
docker compose up -d --build
```

Le VPS va télécharger Node.js, compiler l'application Vite (ce qui prend un peu de temps), puis lancer Caddy.

Étape 4 : Générer la clé SSH pour le futur CI/CD

Une fois l'application en ligne, on prépare le terrain pour la Phase B (l'automatisation) en générant une clé SSH dédiée pour GitHub Actions.

```
# Générer une paire de clés dédiée (sans passphrase)
ssh-keygen -t ed25519 -C "github-actions" -f ~/.ssh/github-actions

# Autoriser la clé publique à se connecter au VPS
cat ~/.ssh/github-actions.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
chmod 700 ~/.ssh

# Afficher la clé PRIVÉE à copier plus tard dans le secret GitHub SSH_PRIVATE_KEY
cat ~/.ssh/github-actions
```

Étape 6 : Vérifier que tout fonctionne

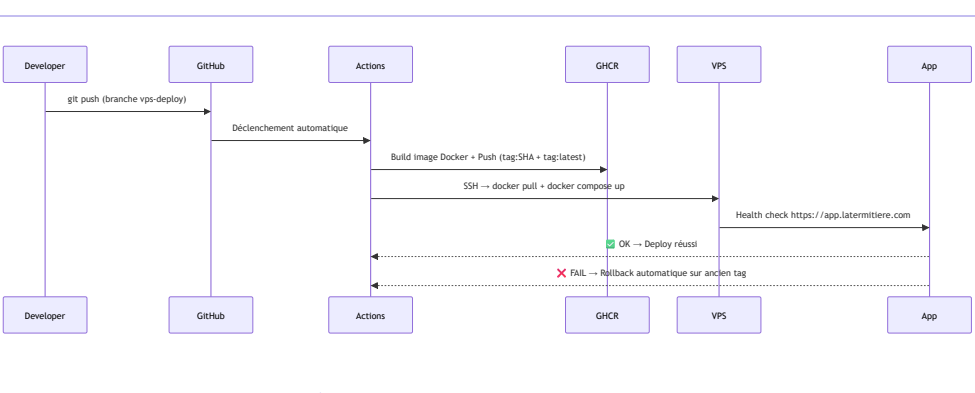
```
docker ps | grep termitiere-web
# → doit afficher le conteneur en status "Up", ports 80 et 443 exposés

curl -s https://app.latermitiere.com
# → doit retourner le HTML de ton app React
```

✅ **Résultat attendu :** L'application est accessible sur `https://app.latermitiere.com` avec un certificat SSL valide géré automatiquement par Caddy.

7.3 PHASE B — DÉPLOIEMENT AUTOMATIQUE (CI/CD via GitHub Actions)

Une fois le déploiement manuel validé, le pipeline GitHub Actions prend le relais pour **tous les déploiements suivants**. Plus besoin de se connecter manuellement au VPS.

7.3.1 Pipeline : Flux complet


Developer
GitHub
Actions
GHCR
VPS
App
git push (branche vps-deploy)
Déclenchement automatique
Build image Docker + Push (tag:SHA + tag:latest)
SSH → docker pull + docker compose up
Health check https://app.latermitiere.com
✅ OK → Deploy réussi
❌ FAIL → Rollback automatique sur ancien tag

7.3.2 Secrets GitHub à configurer

Dans **Settings → Secrets and variables → Actions** du dépôt `termitiere-platform` :

Secret	Valeur
`VPS_HOST`	`31.207.37.96`
`VPS_USER`	`bawa`
`SSH_PRIVATE_KEY`	Contenu de `~/.ssh/github-actions` (clé privée Ed25519)
`GHCR_TOKEN`	PAT GitHub avec permission `read:packages`
`VITE_SUPABASE_URL`	`https://api.latermitiere.com`
`VITE_SUPABASE_ANON_KEY`	La clé `anon` générée par Supabase

7.3.3 Rollback Automatique (Filet de sécurité)

Le script de déploiement intègre un rollback automatique. En cas d'échec du health check :

1. Le script sauvegarde le tag de l'image actuellement en production.
2. Il déploie la nouvelle image.
3. Il teste `curl -f -s https://app.latermitiere.com`.
4. **Si ça plante → il restaure immédiatement l'ancienne image** sans intervention manuelle.

7.3.4 Deploy manuel d'urgence (si GitHub Actions est down)

⚠️ **Sécurité :** Ne jamais écrire le token en clair dans le terminal.

```
# Se connecter au VPS
ssh bawa@31.207.37.96
cd ~/termitiere-platform

# Login GHCR
echo "ghp_TON_PAT" | docker login ghcr.io -u ton-username --password-stdin

# Pull le SHA voulu
docker pull ghcr.io/la-termitiere/termitiere-platform:<SHA>

# Définir le tag à déployer
echo "IMAGE_TAG=<SHA>" >> deploy/.env.deploy

# Redémarrer le conteneur
docker compose \
  --env-file deploy/.env.deploy \
  -f deploy/docker-compose.prod.yml \
  up -d --force-recreate
```

---

8. CONFIGURATION FRONTEND (.env)

```
DOMAIN_APP=app.latermitiere.com
DOMAIN_API=api.latermitiere.com
VITE_SUPABASE_ANON_KEY=xxxx
```

---

9. DOCKER COMPOSE FRONTEND (IMPORTANT)

⚠️ Toujours utiliser :

```
docker compose up -d --force-recreate
```

❌ éviter :

```
docker compose down # (provoque un downtime inutile)
```

---

10. DNS CONFIGURATION

```
app.latermitiere.com → IP VPS
api.latermitiere.com → IP VPS
```

---

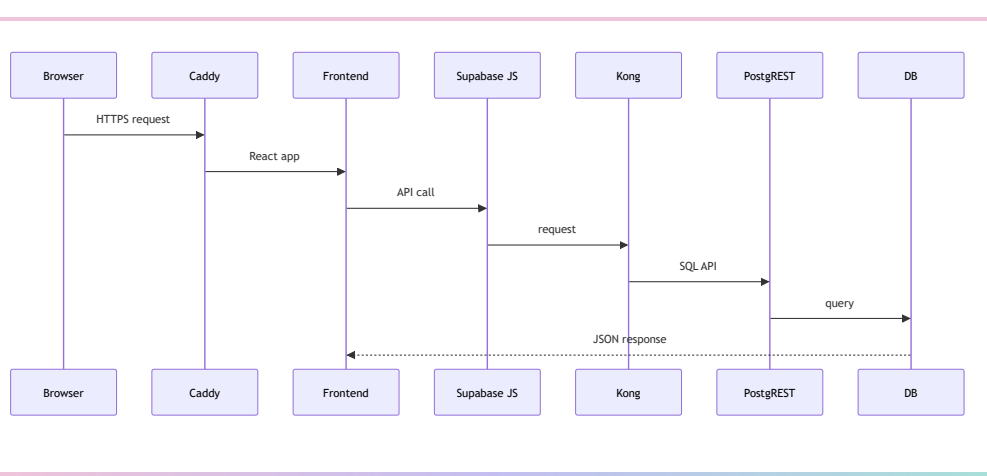
11. HTTPS (CADDY)

Caddy gère automatiquement :

- SSL Let's Encrypt
- HTTPS
- Reverse proxy frontend
- Proxy API vers Kong

---

12. FLUX RÉEL REQUÊTE



---

13. POINT CRITIQUE (TRÈS IMPORTANT)

FRONTEND ≠ BACKEND

Frontend	Backend
UI React	Supabase
GHCR deploy	Docker stateful
stateless	database persistante

---

14. PROBLÈMES COURANTS & SOLUTIONS DEVZ

❌ Données ne changent pas

vérifier :

```
docker compose ps
curl http://localhost:8000
docker logs supabase-storage
```

❌ Storage ou Realtime unhealthy

👉 Ce n'est PAS NORMAL.

- Si tu es sur 4GB de RAM : c'est un problème de mémoire (OOM).
- **Solution :** Ajoute un Swap de 2GB sur le serveur.

```
fallocate -l 2G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
echo '/swapfile none swap sw 0 0' | tee -a /etc/fstab
```

---

15. RESTART BACKEND (SAFE)

```
cd /home/bawa/supabase/docker

docker compose down --remove-orphans
docker compose up -d
```

---

16. BONNES PRATIQUES

- jamais exposer PostgreSQL sur `0.0.0.0`
- jamais commit .env
- toujours utiliser SHA tags
- rollback via image précédente
- backup DB régulier avec cron et `pg_dump`

---

17. CI/CD FUTUR (BACKEND)

À venir :

- versioning Supabase
- migration automatique SQL
- backup PostgreSQL auto
- monitoring (Grafana)
- alerting healthcheck

---

18. MODE DIAGNOSTIC RAPIDE

```
docker ps
docker compose ps
docker logs <container>
free -m
df -h
dmesg -T | grep -i oom # Voir si un container a crashé par manque de RAM
```

---

19. RÉSUMÉ FINAL

✓ Frontend = CI/CD complet
✓ Backend = stateful manuel
✓ DB = persistante
✓ API = Kong
✓ Proxy = Caddy

🚀 FIN DU GUIDE